KernAdapt

面向混合负载的自适应
内核资源管控 Agent

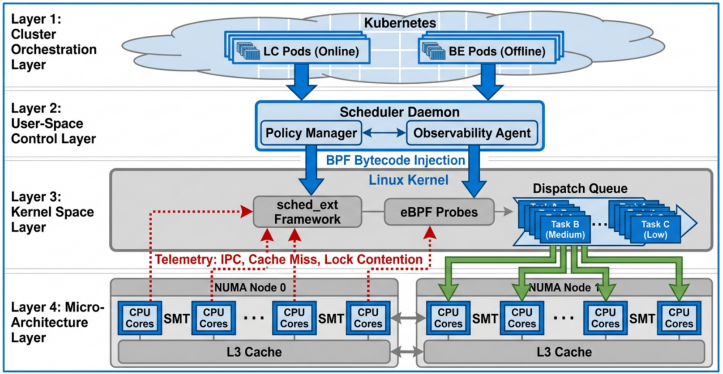
OSKernel

1.1 背景挑战：传统 Linux 调度器的瓶颈

混合负载对吞吐、响应时间和资源分配提出不同要求

不同负载，不同需求

- AI 推理的吞吐需求
- Agent 任务的多步工具调用
- CI/CD 与微服务的并发执行
- 数据库对事务尾延迟敏感



调度策略需要适应负载变化，也需要满足不同应用的性能目标。

1.1 不同应用，共同的调度难题

任务依赖、数据位置与负载变化，共同影响应用完成时间

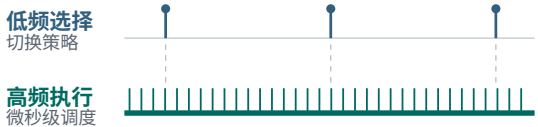
让调度直接服务于应用任务的完成。

1.2 技术挑战：调度器与大模型结合的瓶颈

大模型难以直接参与微秒级调度，细粒度优化仍需内核算法

01 大模型参与策略选择

大模型分析负载、评估策略 → 轻量模型选择已有策略



大模型推理成本高，响应慢。
微秒级调度，不能逐次等待大模型。

细粒度优化，需要新的内核算法

02 生成策略代码

机制知识与实验记录
↓ 提供依据
大模型生成策略代码
↓ 比较效果
运行检查与性能实验

代码能运行，仍需验证：
机制为何有效，何时适用。

有效策略，需要实验验证

研究切入点 原创内核机制 + 实验知识 → **支持 Agent 选择与配置**

1.3 总体研究路线

提出调度机制，通过实验形成知识，再研究 Agent 如何选择与配置

原创机制提供新动作

锁协同调度：加快锁交接
时间片 + CPU 放置
围绕同一把锁，联合调整

任务协同调度：缩短工具等待
优先推进被等待的进程
CPU 用量仍计入所属会话

实验知识提供依据

对照实验　消融实验　干扰实验

适用条件　参数效果　收益与代价

Agent 决策选择与执行

KernAdapt：结合应用状态与目标

读取状态 → 预测组合效果 → 检查并执行 → 验证与记录

研究路线　原创机制 → 实验知识 → **Agent 选择、配置与执行**

第二章　锁协同调度（LockCoop）

围绕锁交接，改进时间片分配与 CPU 放置

01　总体介绍

02　锁协同调度

03　任务协同调度

04　KernAdapt

05　总结与推广

本章内容

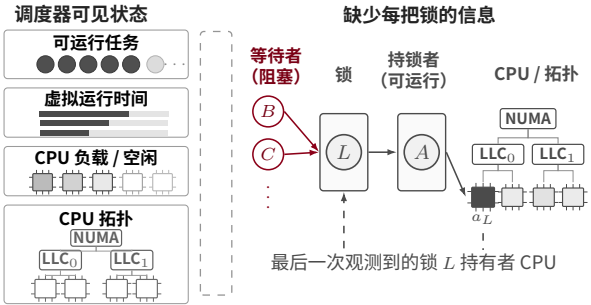
2.1　锁交接问题与逐锁状态

2.2　解锁加速、就近唤醒与机制创新

2.3　机制验证与跨平台实验

2.1 锁协同调度：调度器缺少锁交接信息

持锁者调度延迟会增加同一把锁上其他线程的等待时间



解锁前

持锁者未获 CPU，等待者只能继续等待。

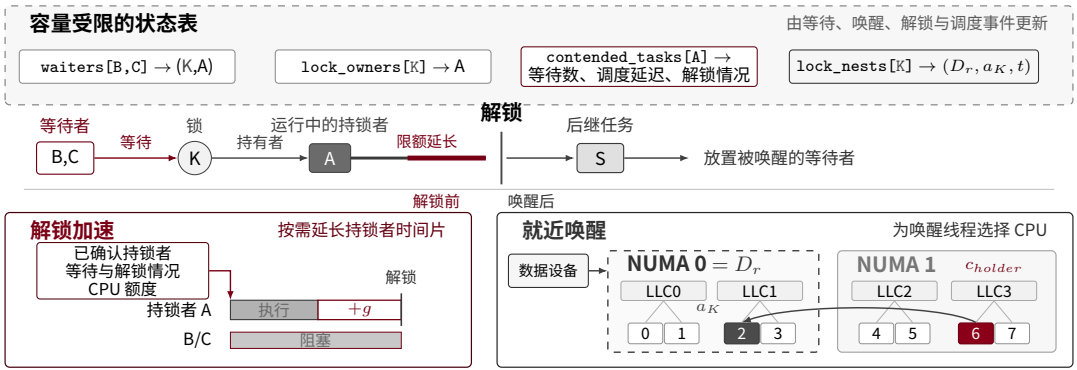
唤醒后

跨核执行可能增加共享数据的缓存传输开销。

同时回答：谁需要先运行，唤醒后放在哪个 CPU。

2.1 逐锁记录，同时改进时间片和 CPU 放置

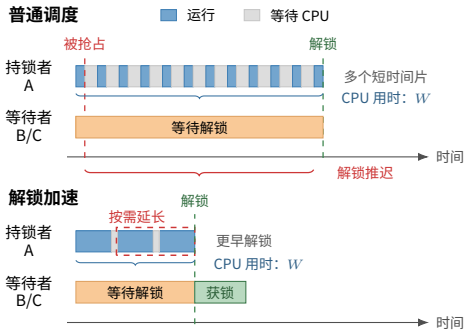
每把锁保存等待者、已确认持锁者，以及持锁者最近运行的 CPU

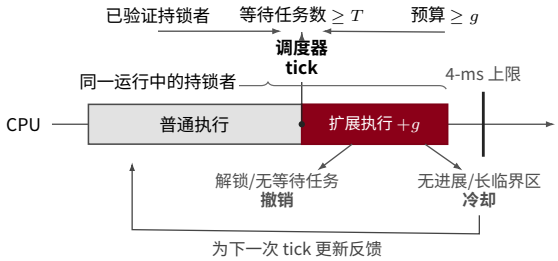


解锁加速延长持锁者的时间片；就近唤醒为唤醒线程选择合适的 CPU。

2.2 解锁加速（Boost）：按需延长持锁线程时间片

结合等待线程数、持锁者调度延迟和解锁进展，按需延长时间片





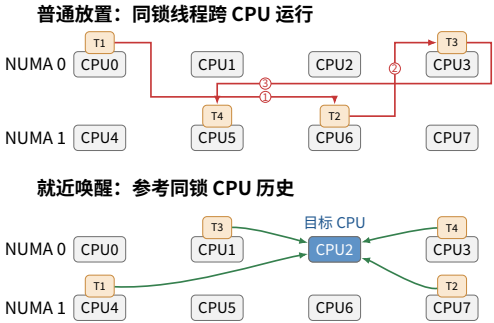
确认持锁者
只为身份已确认、正在运行的持锁者延长时间片。

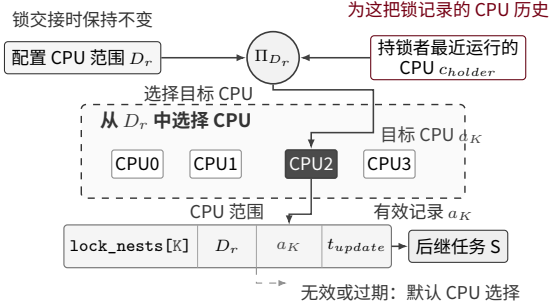
限制额度
任务、锁和 cgroup 分别记账，单次延长设有上限。

及时停止
解锁、额度用完或迟迟不解锁时，停止延长。

2.2 就近唤醒（Local）：利用同锁 CPU 执行历史

首次执行在指定 CPU 范围内分配；后续唤醒参考同锁持有者的运行位置





减少共享数据的跨核传输。记录失效或 CPU 不可用时，采用默认 CPU 选择。

2.2 解锁加速与就近唤醒：两处机制改进

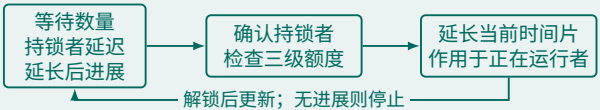
锁竞争反馈控制时间片延长，同锁执行历史指导 CPU 选择

RSEQ：程序申请延长

进入临界区 → 请求延长

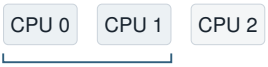
由用户态显式发起请求

LockCoop・Boost：按锁状态与进展判断

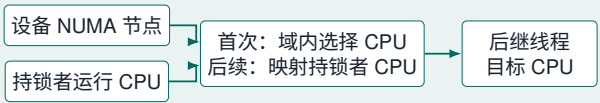


Nest：复用近期活跃 CPU

记录哪些 CPU 最近被使用



LockCoop・Local：保留同锁 CPU 执行历史



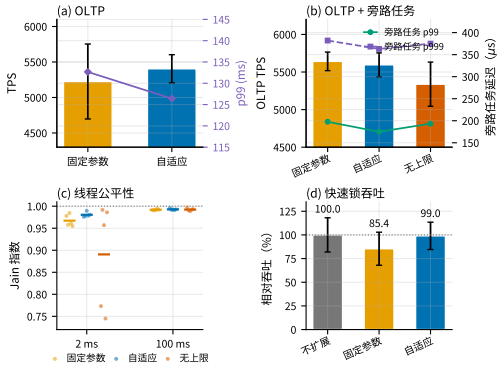
同一把锁的状态用于两次决策：解锁前延长时间片，唤醒后选择 CPU。

RSEQ 时间片扩展： Linux Kernel Documentation, Scheduler Time Slice Extensions. Linux 7.0, 2026.
Nest： Lawall et al. OS Scheduling with Nest: Keeping Tasks Close Together on Warm Cores. EuroSys, 2022.

2.2 锁协同调度：自适应参数与控制效果

按锁竞争状态调整阈值和时间片，并限制额外 CPU 时间

控制项	策略	取值与限制
等待阈值	随竞争与进展调整	初值 4 范围 2–16
时间片	按状态选择档位	1 / 2 ms 上限 4 ms
扩展预算	任务 / 锁 / cgroup 限额	20 / 40 / 40% 窗口 100 ms
CPU 放置	使用同锁 CPU 执行历史	有效期 2 s



相对固定参数配置（保留扩展上限）：吞吐 +3.43%，p99 延迟 −4.72%。

2.3 实验环境：CPU、内存与系统配置

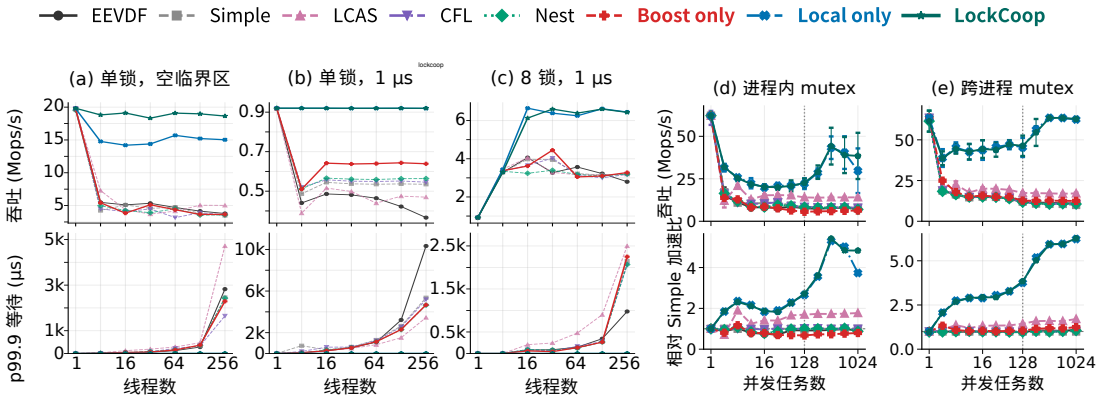
覆盖 ARM64、x86、RISC-V 与 VMware

平台	处理器与 CPU 数量	内存	操作系统 / 内核
ARM64	双路 Altra Max M128-30 **256 核**，无超线程	**1 TiB**	openEuler 24.03-LTS（实验平台） 默认内核：Linux 6.6 实验内核：Linux 6.17（补丁适配）
x86	双路 Xeon Platinum 8375C **64 核 / 128 线程**	**256 GiB**	Ubuntu 24.04 Linux 6.17
RISC-V	Milk-V Pioneer（C920） **64 核**，4 个 NUMA 节点	**128 GiB**	RevyOS Linux 6.17.13
VMware	EPYC 9654 虚拟 CPU **32 vCPU**，单 vNUMA 节点	**128 GB**	Ubuntu 26.04 Linux 7.0

同一平台、相同 CPU 范围与应用配置，对照比较调度效果。

2.3 互斥锁实验：临界区长度、锁数量与共享范围

左：单锁与多锁的吞吐、等待时间；右：进程内与跨进程互斥锁的吞吐、加速比

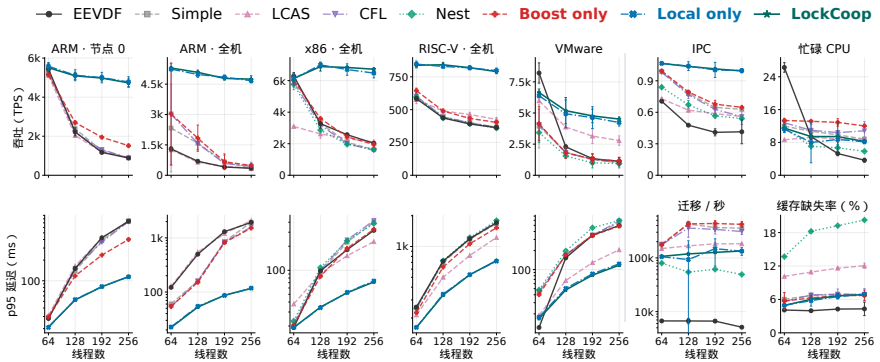


256 线程、单锁空临界区：锁协同调度为 18.64 Mops/s，是 scx_simple 的 5.36 倍。右侧竖虚线标记 128 个在线 CPU。

LCAS：Cui et al., TACO 2013. **CFL**：Park and Eom, PPoPP 2024. **Nest**：Lawall et al., EuroSys 2022.
EEVDF：Stoica and Abdel-Wahab, Technical Report 1995；Linux EEVDF 文档. **Simple**：Linux scx_simple，sched_ext 示例实现.

2.3 多平台 OLTP：吞吐提高，尾延迟降低

MariaDB OLTP，64–256 线程；左：多平台性能，右：x86 硬件计数器

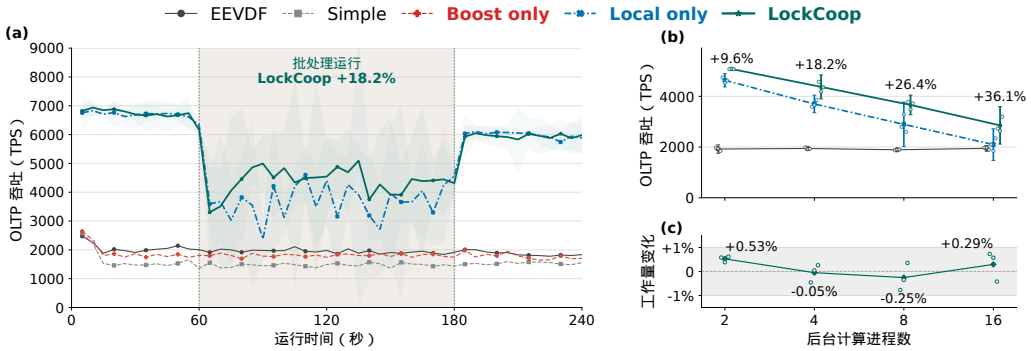


256 线程，相对 scx_simple **吞吐 2.15–13.9 倍；p95 尾延迟降低 56.6%–94.0%**

LCAS：Cui et al., TACO 2013. **CFL**：Park and Eom, PPoPP 2024. **Nest**：Lawall et al., EuroSys 2022.
EEVDF：Stoica and Abdel-Wahab, Technical Report 1995；Linux EEVDF 文档. **Simple**：Linux scx_simple，sched_ext 示例实现.

2.3 存在 CPU 竞争时，解锁加速带来额外收益

完整方案对比仅启用就近唤醒，分析延长时间片的作用



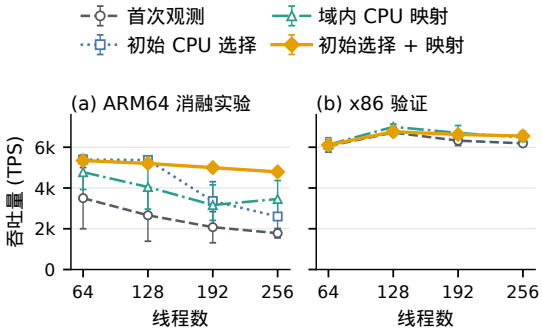
吞吐再提高 9.6%–36.1%；并发批处理完成的工作量差异不超过 0.53%。

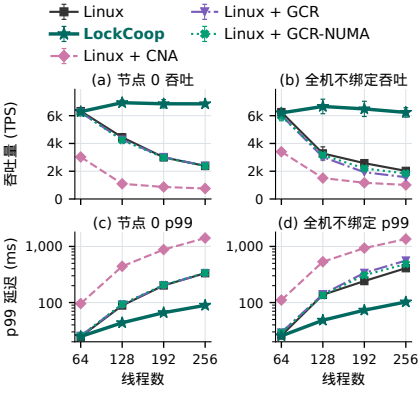
分别改变 CPU 密集后台任务数量。

EEVDF：Stoica and Abdel-Wahab, Technical Report 1995；Linux EEVDF 文档. **Simple**：Linux scx_simple，sched_ext 示例实现.

2.3 CPU 放置机制验证与锁实现对比

左：初始 CPU 选择与域内映射消融；右：锁实现与 NUMA 优化方法对比





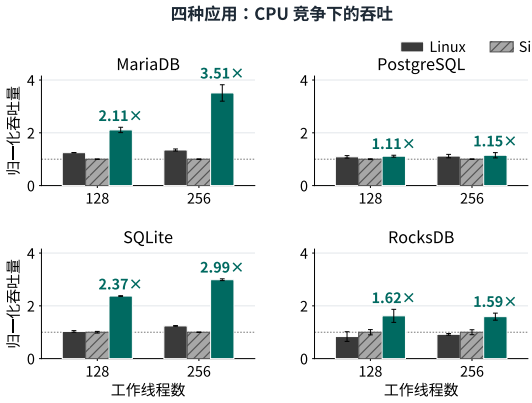
ARM64 全机：吞吐为首次观测方案的 2.69 倍
变异系数由 18.19% 降至 4.41%。

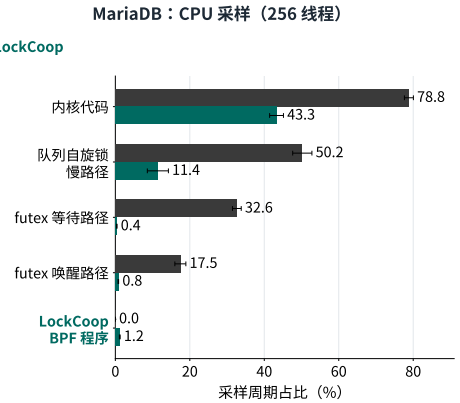
误差线为 95% 置信区间；右侧 Linux 与锁协同调度使用原生 pthread mutex，p99 纵轴为对数刻度。

CNA：Dice and Kogan. Compact NUMA-Aware Locks. EuroSys, 2019.
GCR / GCR-NUMA：Dice and Kogan. Avoiding Scalability Collapse by Restricting Concurrency. Euro-Par, 2019.

2.3 不同应用与 CPU 干扰

MariaDB、PostgreSQL、SQLite 与 RocksDB，128 / 256 个工作线程



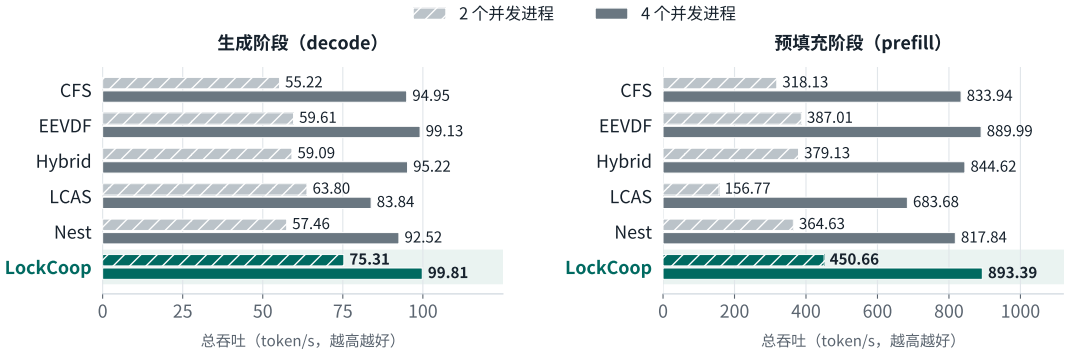


x86 不绑核。左：相对 scx_simple 归一化。右：MariaDB CPU 采样占比，调用路径可重叠。三轮均值 ± 标准差。

Simple：Linux scx_simple，sched_ext 示例实现.

2.3 AI 推理吞吐对比

llama.cpp / Qwen3-0.6B；2、4 个并发进程；每项为 5 次运行均值



2 并发相对 EEVDF：生成吞吐 +26.34%，预填充吞吐 +16.45%；4 并发时与 EEVDF 接近。

LCAS： Cui et al., TACO 2013. **Nest：** Lawall et al., EuroSys 2022.
EEVDF： Stoica and Abdel-Wahab, Technical Report 1995；Linux EEVDF 文档. **CFS：** Linux Kernel Documentation, CFS Scheduler.

2.3 锁协同调度小结：多平台、多应用的性能收益

围绕锁交接，协同调整持锁者时间片与后继线程 CPU 放置

同一把锁
持锁者 · 等待者 · CPU 历史

→

联合调整时间与位置
解锁加速 + 就近唤醒

→

提供给 Agent 的配置参数
等待阈值 · 时间片上限 · CPU 范围

多平台 OLTP：吞吐提高，尾延迟降低

2.15–13.9× 吞吐

p95 延迟降低 **56.6%–94.0%**

CPU 竞争：两项机制相互配合

+9.6%–36.1% 吞吐

批处理工作量差异 ≤ **0.53%**

四类数据库：均有吞吐收益

1.11–3.51× 吞吐

MariaDB / PostgreSQL / SQLite / RocksDB

AI 推理：生成与预填充吞吐提高

+26.34% / +16.45%

生成阶段 / 预填充阶段

机制创新 逐锁反馈：时间片 + CPU 选择 → **Agent 可配置的调度机制**

第三章　任务协同调度（TaskCoop）

识别工具等待关系，兼顾工具响应与会话公平性

01　总体介绍

02　锁协同调度

03　任务协同调度

04　KernAdapt

05　总结与推广

本章内容

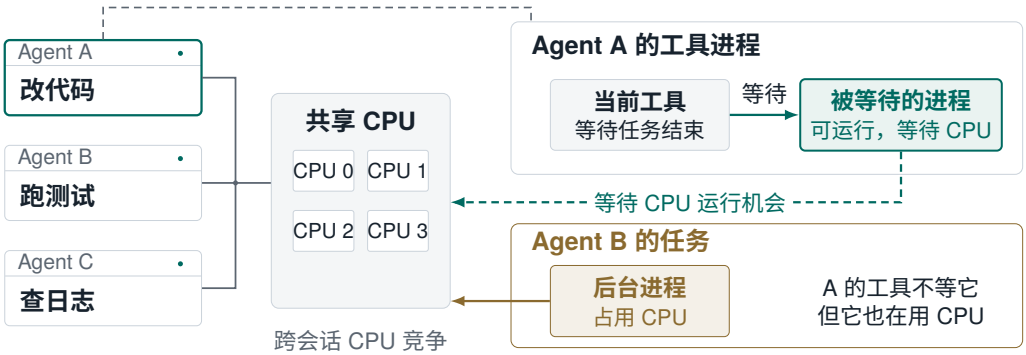
3.1　多 Agent 的工具等待问题

3.2　关键进程调度与会话公平

3.3　自建任务基准与性能验证

3.1 多 Agent 并发：工具等待与 CPU 竞争

B 的后台任务争用 CPU，拖慢 A 当前工具所等待的进程



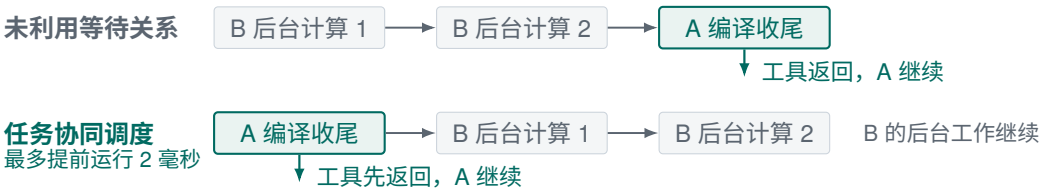
其他 Agent 的后台任务，可能拖慢当前工具所等待的进程。

3.1 举个例子：编译场景

Agent A 等待编译结果，Agent B 的后台进程仍在竞争 CPU

任务协同调度沿等待链，找到编译器

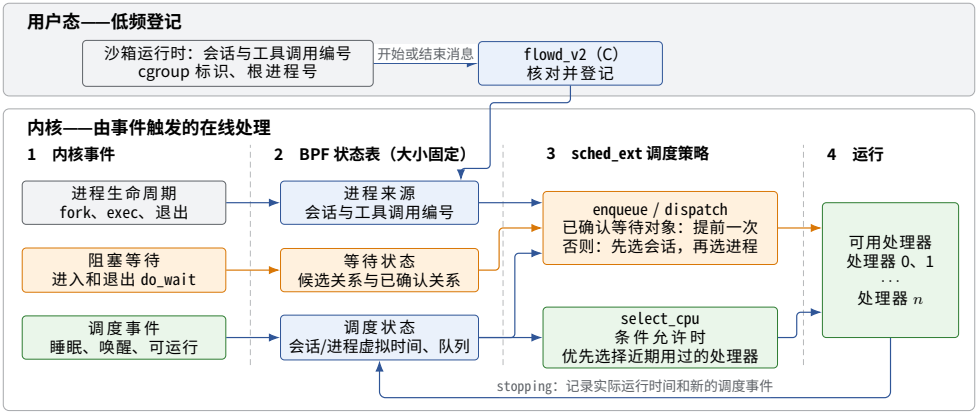
CPU 用量仍计入所属会话



先推进真正影响工具返回的进程，让 Agent 尽早继续下一步。

3.2 运行时登记会话，内核识别实际等待

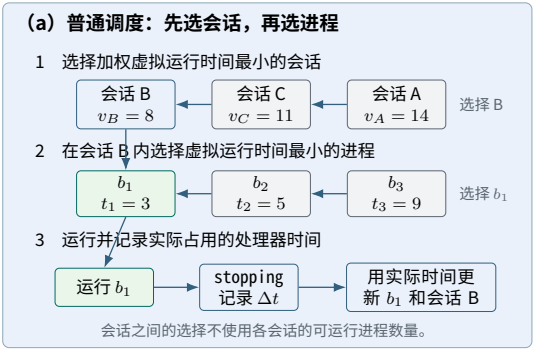
每个会话使用独立 cgroup；C 程序登记，BPF 按事件更新状态

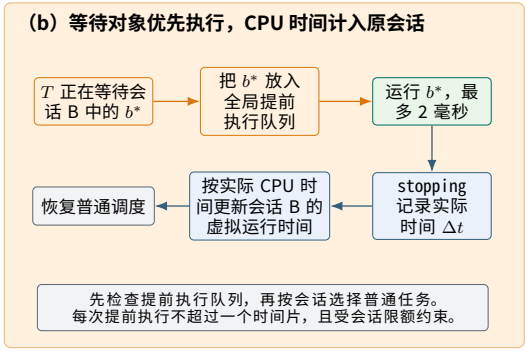


C 程序登记会话与工具编号；BPF 跟踪进程创建、阻塞、唤醒和退出。

3.2 等待进程优先调度，会话之间公平分配

每个会话使用独立 cgroup；提前执行的 CPU 时间仍计入所属会话

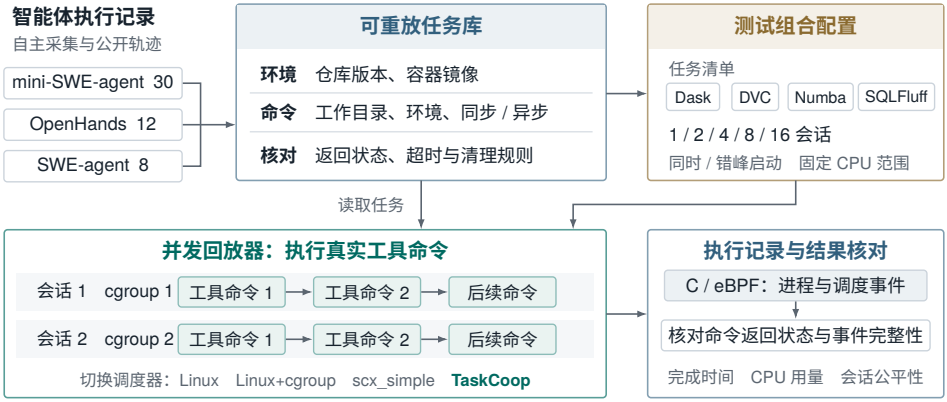




提前执行最长 2 毫秒；实际使用的 CPU 时间仍计入所属会话。

3.3 HarnessReplayBench：自建多 Agent 任务基准

50 个任务、42 个代码仓库、2,940 条工具命令；统一环境下重复执行

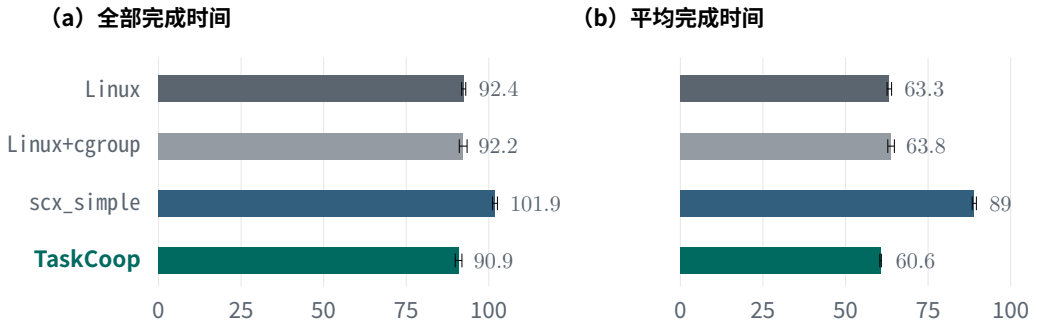


测量工具在主机上的执行时间，不包含模型推理和远程 API 等待。

Linux+cgroup：Linux Kernel Documentation, Control Group v2: CPU. **Simple**：Linux scx_simple，sched_ext 示例实现.

3.3 四会话实验：平均完成时间缩短 4.2%

四会话混合工作流；每种方案交错运行五轮



完成时间（秒，越低越好）；误差线为样本标准差。

平均完成时间：相对 Linux 缩短 4.2%，相对 scx_simple 缩短 31.9%。

Linux+cgroup：Linux Kernel Documentation, Control Group v2: CPU.　**Simple**：Linux scx_simple，sched_ext 示例实现.

3.3 八工作流：平均完成时间缩短 7.32%

4 个 CPU；每种方案交错运行三轮

方案	平均工作流完成时间	全部工作流完成时间
原生 Linux	25.986 ± 0.100 秒	52.201 ± 0.497 秒
TaskCoop	24.084 ± 0.016 秒	53.584 ± 0.057 秒
启用跨命令等待原型	24.103 ± 0.078 秒	53.587 ± 0.053 秒

−7.32%
平均完成时间

+2.65%
全部完成时间

3 轮
均值 ± 样本标准差

以全部完成时间增加 2.65% 为代价，换取平均完成时间缩短 7.32%。

3.3 任务协同调度小结：缩短工作流平均完成时间

依据实际等待关系调整执行次序，CPU 用量仍计入所属 cgroup

确认工具的实际等待
阻塞、唤醒与退出事件

→

被等待进程提前运行
会话预算内，单次最多 2 毫秒

→

CPU 用量仍计入所属会话
更新所属会话的虚拟运行时间

自建多 Agent 任务基准 HarnessReplayBench

50 个任务　**42** 个代码仓库　**2,940** 条工具命令

四会话：两组任务组合均有收益

平均完成时间缩短

4.2%–5.2% 相对 Linux

相对 scx_simple 缩短 **27.9%–31.9%**

两组四会话组合；每种方案交错运行五轮。

八工作流：平均耗时降低，总完成时间略增

平均完成时间缩短

7.32% 相对 Linux

全部完成时间增加 2.65%

4 个 CPU、8 个工作流；每种方案交错运行三轮。

机制创新　等待关系识别 → 优先执行 → **缩短 Agent 工具等待**

评估主机工具执行，不含模型推理与远程 API 等待。

Simple：Linux scx_simple，sched_ext 示例实现.

第四章　KernAdapt

利用机制知识与实验结果，研究参数选择和执行反馈

01　总体介绍

02　锁协同调度

03　任务协同调度

04　KernAdapt

05　总结与推广

本章内容

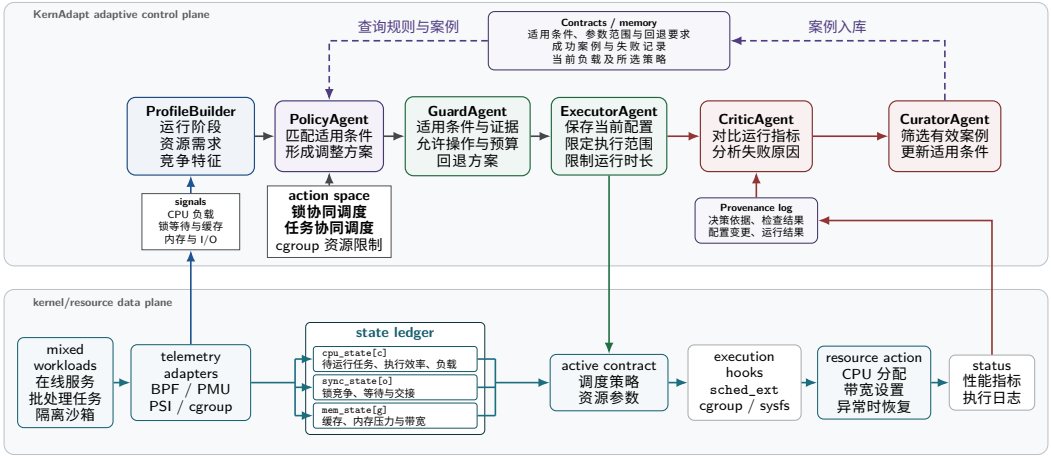
4.1　机制知识与决策框架

4.2　参数组合选择与风险控制

4.3　执行验证与经验更新

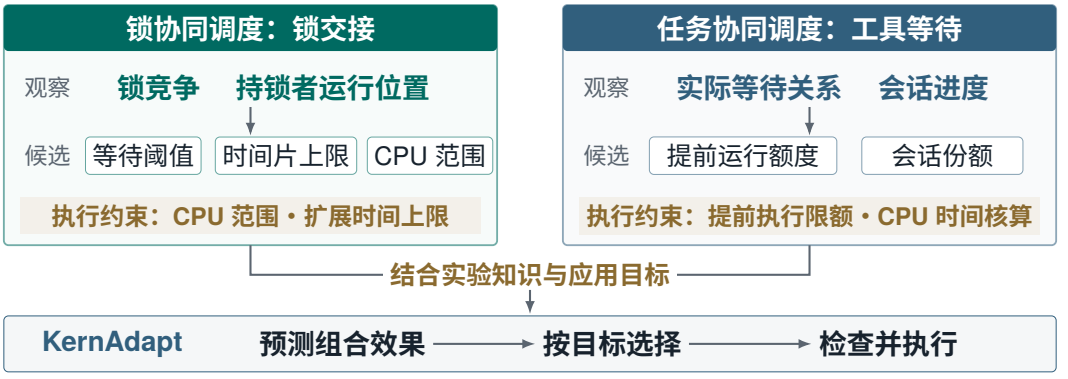
4.1 自进化调度策略框架：KernAdapt

机制知识与执行记录支持决策，独立模块负责检查、执行和恢复



4.1 KernAdapt：何时使用，怎样配置？

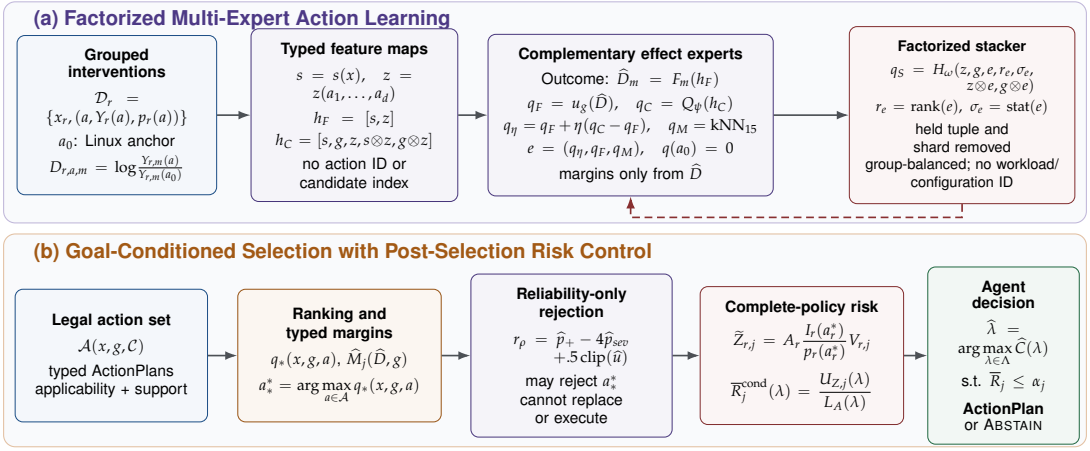
以锁交接和工具等待为例，明确候选参数与执行约束



机制与决策 机制提供动作与使用依据 → Agent 决定何时用、怎样配

4.2 KernAdapt 的完整学习与决策算法

上：学习动作与组合效果；下：按目标选择方案并检查风险

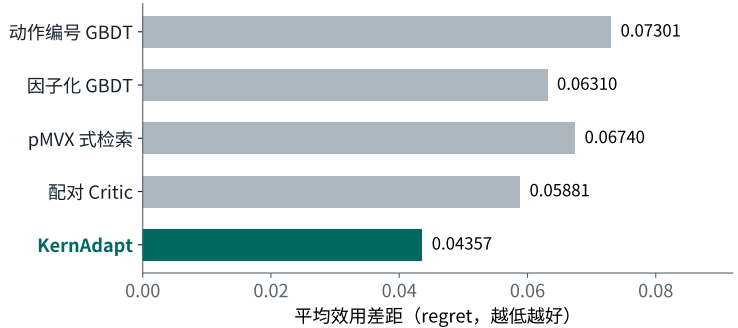


用独立数据校准完整决策流程；测试期间固定候选生成、排序和接受规则。

4.2 参数组合选择：缩小与已测最佳配置的效用差距

560 组实验，2,800 次执行；8 种完整参数组合，5 个随机种子

按实验组划分训练与测试；被测组合不进入对应训练集，测试期间固定模型。



−30.95%
相对因子化 GBDT

−35.36%
相对 pMVX 式检索

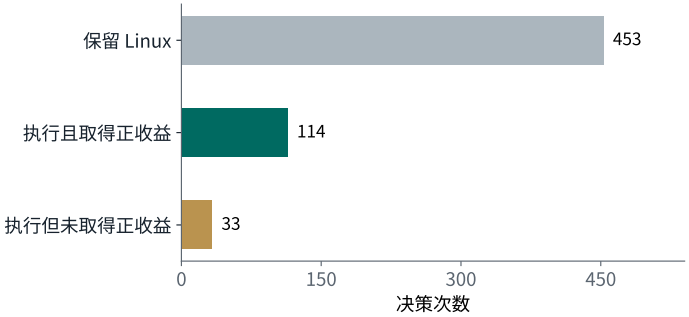
两项效用差距降幅的配对 95% 置信区间均高于零。

各方法使用相同数据与候选集。效用按目标综合性能指标；regret = 本组已测最高效用 − 所选配置效用。

pMVX 式检索参考： Singh et al. pMVX: Policy-Level Multi-Version Execution for Agentic OS Kernel Self-Tuning. AgenticOS Workshop, 2026.
GBDT： Friedman. Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 2001.

4.3 600 次决策中，147 次实际调整资源

固定检索策略选择 scx_adapt 配置；453 次保留 Linux，114 次执行取得正收益



77.55%
取得正收益的执行比例

平均效用增益
0.01398
95% 区间 [0.01094, 0.01694]

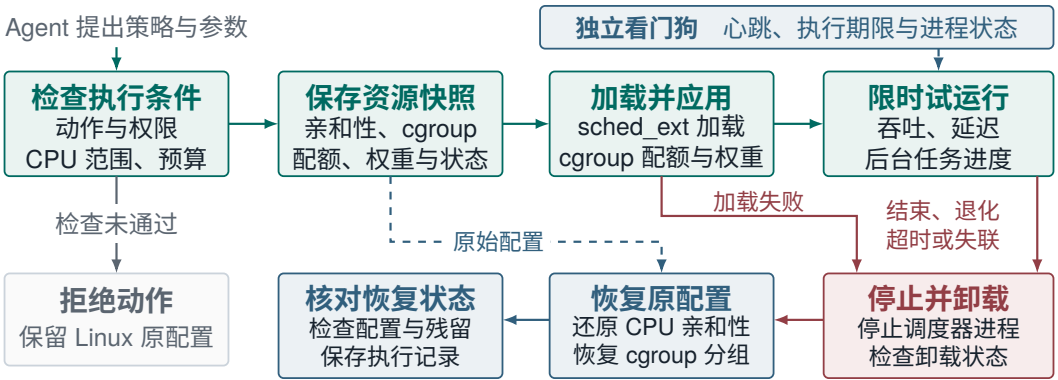
147 次执行的约束结果
未满足约束：2 次
严重退化：未观测到

执行前检查权限、范围与预算；风险过高则保留 Linux，执行后检查状态恢复。

30 轮，每轮 20 个场景；效用按应用目标计算，以 Linux 为零基线。

4.3 动态加载、运行监控与状态恢复

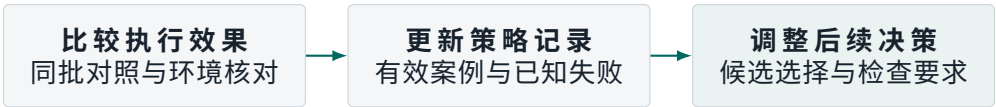
执行器保存配置并应用策略，独立看门狗检查运行期限和异常状态



加载前保存配置，运行中检查状态，退出后恢复资源并核对结果。

4.3 策略如何随着执行经验更新？

保存验证结果与失败原因，更新候选策略状态和后续检查规则



执行中得到的结果	对后续行为的影响
候选通过重复验证	达到验证次数要求后，由候选转为已验证策略
候选反复失败	拒绝候选；已有策略出现失败时补充失败记录
采样或环境异常	延长采集窗口、补充采样，或启用兼容性检查

执行经验改变策略状态与检查要求，为下一次选择提供依据。

第五章　总结与推广

工程适配、机制推广与主要贡献

01　总体介绍

02　锁协同调度

03　任务协同调度

04　KernAdapt

05　总结与推广

本章内容

5.1　项目功能与工程实现

5.2　跨硬件、跨内核与同步支持

5.3　机制推广与主要贡献

5.1 项目功能与工程实现

围绕 Agent 框架、负载感知、策略调整、openEuler 适配和可复现测试

项目功能	已实现的内容	验证依据
Agent 框架与 Skills	Skills 配置、工具调用、权限检查、状态保存与重放	29 / 29 项框架与控制流程检查通过
感知应用负载	采集 CPU 状态、锁等待及应用性能	采集记录与功能测试
调整 CPU 策略	sched_ext 启停、调参与异常恢复	负载切换、加载失败、退化和超时测试
资源管控扩展	eBPF 事件采集；cgroup 份额、权重与额度	会话公平、CPU 记账与恢复测试
openEuler 适配	锁协同调度编译、运行与性能测试	环境配置、运行记录与多平台对照
可复现实验	实验脚本、配置与原始结果	对照实验、机制消融与结果核算

项目代码 **13.1 万行**，含实现、测试与实验脚本，不含空行、注释和配置变体。

5.2 锁与同步支持及跨平台适配

跨硬件架构、跨内核版本适配；支持多种锁与同步机制

7 类锁与同步路径

内核同步 mutex / rwsem	私有 futex pthread mutex
共享 futex	PI futex
futex requeue	pthread rwlock
spin / CAS（需标注）	

锁协同调度：逐锁识别与调度

逐锁状态
锁标识
持锁者、等待者
CPU 执行历史

解锁加速
持锁者时间片延长

就近唤醒
后继线程 CPU 放置

按有效信息启用对应动作

3 类硬件架构 + 虚拟机

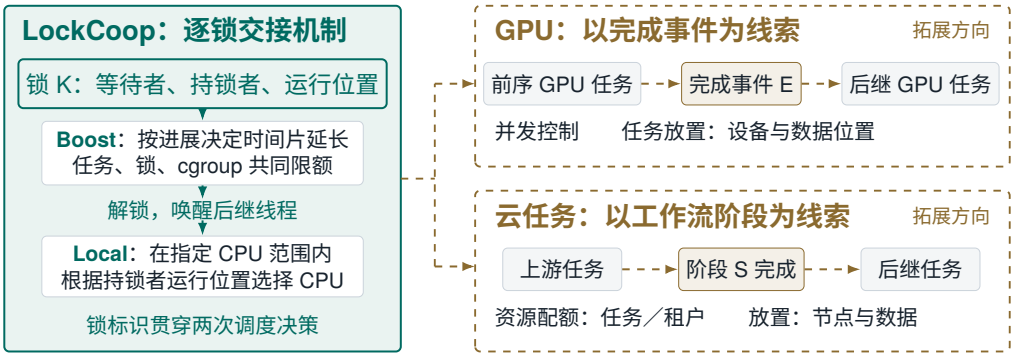
ARM64	x86-64	RISC-V	VMware
openEuler 24.03-LTS（实验平台） 默认内核：Linux 6.6 实验内核：Linux 6.17（补丁适配）	Ubuntu 24.04 Linux 6.17	RevyOS Linux 6.17.13	Ubuntu 26.04 Linux 7.0

项目测试负载：数据库、锁微基准、多 Agent 构建与测试、llama.cpp / Qwen3-0.6B 推理。

跨平台验证以锁协同调度为主。

5.3 从 CPU 调度，探索 GPU 与云任务

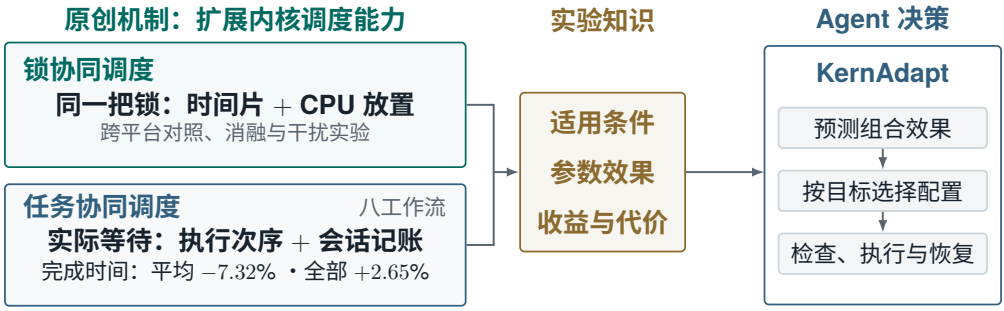
将锁交接中的状态传递方法，拓展到 GPU 任务依赖与云工作流



以依赖关系连接两次决策：前序任务的资源分配与后继任务的放置。

5.3 总结：提出新机制，支持 Agent 决策

从应用等待出发，完成机制设计、实验验证和决策方法研究



资源管控能力 | Agent 决策 | 自适应调参 | 可编程调度

谢谢

KernAdapt

面向混合负载的自适应内核资源管控 Agent

以下为补充实验与实现细节

A.1 不同锁与同步路径的支持

按锁类型识别持锁者和交接关系，分别启用时间片延长与 CPU 放置

锁或同步路径	锁的标识	持锁者与交接信息	可执行的动作
内核 mutex / rwsem	内核对象	持锁者 / 写者	时间片延长 + CPU 放置
私有 futex / pthread mutex	地址空间 + 地址	已校验的持锁者字段	时间片延长 + CPU 放置
共享 futex	内核 futex 键	等待与唤醒关系	CPU 放置
PI futex	内核 futex 键	内核确认的持锁者	时间片延长 + CPU 放置
Futex requeue	源 / 目标 futex 键	等待队列转移	CPU 放置
pthread rwlock	地址空间 + 地址	已确认的写者 / 读者集合	仅为已确认写者延长时间片
spin / CAS（需标注）	作用域 + 锁键	可选持锁者 TID	按有效信息启用动作

A.2 锁信息缺失与记录失效的处理

分别检查持锁者信息与 CPU 执行记录，信息缺失时停用对应优化

运行时情况	处理方法
持锁者无法确认	不延长时间片；有效 CPU 执行记录仍可用于选择 CPU。
私有锁地址相同	用地址空间与地址共同标识锁，分别维护记录。
地址复用或记录过期	清理旧关系，拒绝过期的持锁者与 CPU 记录。
状态表已满或记录缺失	淘汰旧记录；信息缺失时，任务继续参与普通调度。

解锁加速与就近唤醒分别判断；一项停用，不影响另一项。

A.2 锁标识与生命周期验证

检查锁标识是否正确，以及旧记录能否及时清理

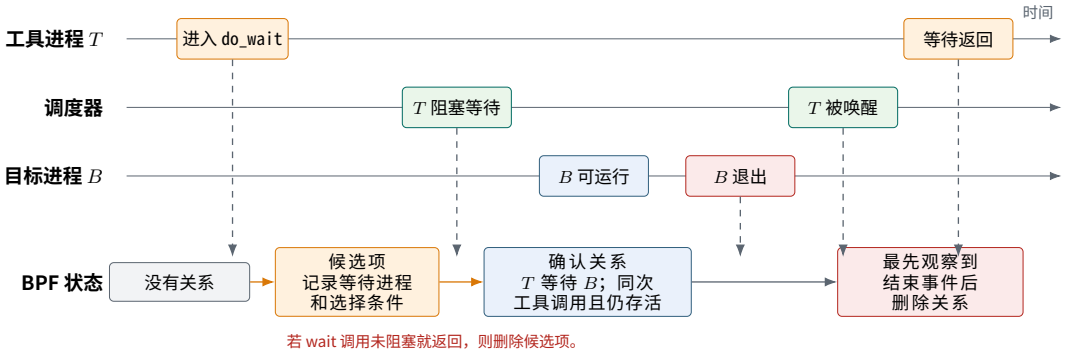
测试对象	验证结果
私有锁地址	不同进程的相同虚拟地址使用不同锁标识
共享锁映射	同一共享锁的不同映射使用同一记录
优先级继承锁	等待、获取、释放各 2,399 次，持锁者识别无遗漏
同地址复用	重新初始化后拒绝旧 CPU 记录
容量与多热点	1,152 个锁标识与 256 把热点锁完成测试，无任务停滞

验证中未出现 BPF 校验失败、状态更新失败或可运行任务停滞。

锁协同调度补充附录：实现验证；CPU 历史记录表上限为 1,024 项。

B.1 确认 wait 调用是否进入阻塞状态

结合阻塞事件与子进程状态，确认工具进程的等待对象



等待返回、进程退出或切换工具后，及时删除旧关系。

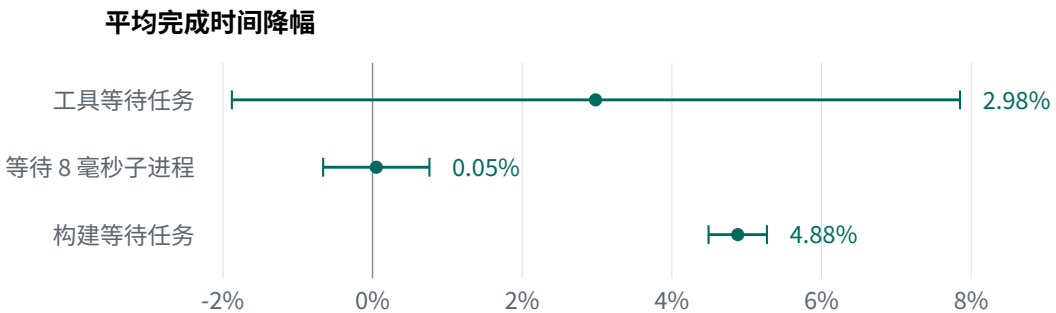
B.1 限制提前运行，准确记录 CPU 用量

处理短等待、权重变化和主动让出 CPU 三种情况

- **短等待：**短计算线程提前运行的额度，不超过本次睡眠时间。
- **权重变化：**变化前后的 CPU 时间，分别按对应权重记账。
- **主动让出：**重新检查队列，安排可运行任务，避免遗漏。
- **Linux 对照：**测试前确认自定义调度器和控制器已经退出。

B.2 等待关系识别与调度的消融实验

关闭等待识别、提前执行及相关优化，保留会话间 CPU 分配，进行对照

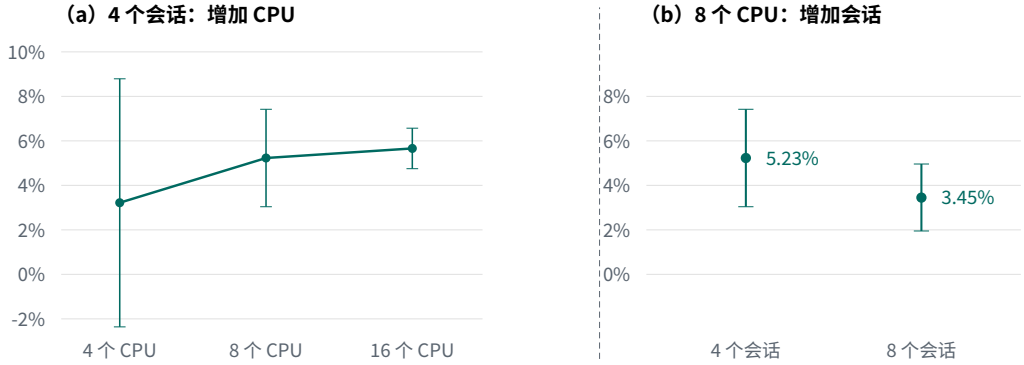


正值表示耗时缩短；误差线为 95% 置信区间。

构建等待任务的平均完成时间缩短 4.88%；短等待任务基本持平。

B.2 任务协同调度的 CPU 与会话规模实验

工具等待任务组合；比较不同 CPU 与会话数量下的平均完成时间



平均完成时间相对 Linux 的降幅；误差线为 95% 置信区间。

每个设置三轮；误差线为逐轮配对 95% 置信区间。

B.2 任务协同调度的运行开销与异常恢复

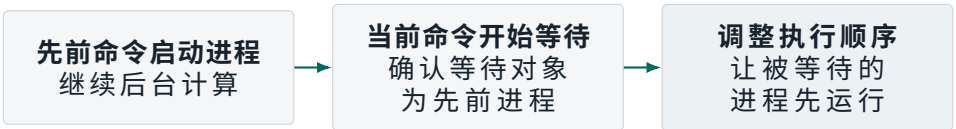
BPF 处理高频事件；登记程序失联超时后，停用提前执行

项目	测量或设置	含义
完整 BPF 处理	CPU 占用率 1.173% （按 8 个 CPU 归一化）	完整路径的 CPU 开销
C 登记程序	峰值内存 8.8 MiB	用户态负责登记与 cgroup 核对
提前执行额度	每 100 毫秒最多 10 毫秒	限制每个会话提前执行的 CPU 时间
登记超时	2 秒	失联后取消对应提前执行状态

开销包括会话调度与等待关系跟踪，以 8 个 CPU 的总可用时间为分母。

B.3 跨命令等待：当前工具等待先前命令的进程

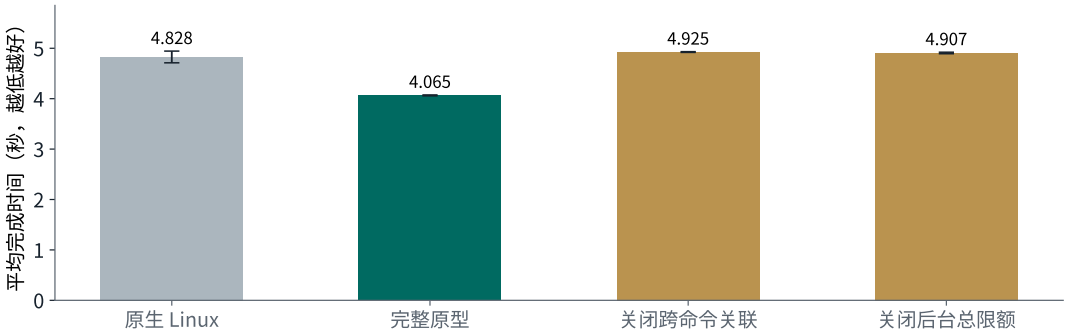
等待对象变化后，调整同一会话内的执行顺序



- 后台进程共享 CPU 时间限额，进程数量不改变总限额。
- 只有确认新的等待关系后，才调整队列中的位置。

B.3 跨命令等待原型：等待识别与后台限额的消融

4 个 CPU、8 个后台进程；固定工作量，每个方案三轮



本组实验中，关闭等待关系更新或后台总限额后，收益消失。

误差线为样本标准差；包含后台工作和 2 秒模拟非 CPU 阶段，不包含应用启动成本。

C.1 KernAdapt 的实验环境与统计方法

补充任务配置、配对方式与重采样方法

项目	设置与方法
CPU 与内核	32 个 AMD EPYC 9654 虚拟 CPU，单 NUMA 节点；Linux 7.0.0-29
任务	schbench、futex mutex/ring、MariaDB 只读 OLTP、RocksDB Get/Scan、TinyLlama 1.1B CPU 推理
配对方式	同一任务条件下运行候选方案与 Linux，随机安排执行顺序
组合比较	随机种子为 17、23、31、47、59；先在同一实验组内求均值，再跨组比较
实机效用	以每轮 20 个场景为整体，进行 10,000 次重采样，计算 95% 置信区间

C.2 KernAdapt：风险发生率的统计上界

以实际执行的实验组为统计单位，计算精确单侧置信上界

风险项目	90% 单侧上界
至少一项约束未满足	3.58%
严重性能退化	1.55%

- 使用 Clopper–Pearson 精确二项置信上界。
- 零次观测仍对应非零上界，不能据此认定风险为零。

对应实机执行实验；统计结论限于本机、已有任务类型和测试期间固定的策略。

D.1 尚需验证的场景

三项成果分别完成实验；联合运行的性能仍需同批验证

研究	待验证场景
锁协同调度	更多应用锁实现与硬件拓扑
任务协同调度	更多跨命令等待；模型推理与远程服务等待
KernAdapt	跨机器、未见任务类型与持续在线学习

D.2 构建、演示、测试与结果核算

仓库提供统一命令，配置与原始结果均可查阅

内容	仓库命令	可观察的结果
构建基础程序	`make build`	源码检查与基准程序编译
演示 Agent 流程	`make demo`	策略检查、角色权限、事件记录与状态保存
功能验证	`make test`	各模块的检查结果
核算实验结果	`make verify-results`	openEuler 实验的 140 个样本、28 种实验配置

make demo 使用合成输入演示控制流程；性能结果来自各项真实执行实验。

真实加载需要兼容内核和相应权限。配置、运行方法和结果位于仓库 tests/ 与 docs/。